

01

Robotics & Automation
Codec Technologies India
Internship 2026

Autonomous Warehouse Robot

A*/Dijkstra planning, LiDAR
re-planning, pure pursuit,
OpenCV item picking

AT A GLANCE

6/6

Orders completed

0

Collisions (whole run)

147 m / 150 m

Driven / planned

5

Re-plans (3 boxes)

98 %

Recogniser (50 items)

98 vs 331

A* vs Dijkstra nodes

270 s

Mission time (sim)

17/17

Unit tests

VERDICT

SIM PASS

TESTS 17/17

0 COLLISIONS

DECLARED SUBSTITUTION

on-mission picks were 6/6 because the wanted item only has to beat two distractors.

Leg	Planned (m)	Driven (m)	Time (s)	Re-plans	Pick
ORD01 to_pick	3.21	2.90	6.70	0	True
ORD01 to_drop	15.62	15.27	22.30	0	-
ORD02 to_pick	11.00	10.81	18.20	0	True
ORD02 to_drop	16.24	15.59	27.25	0	-
ORD03 to_pick	10.62	11.23	22.15	2	True
ORD03 to_drop	6.62	6.07	13.25	0	-
ORD04 to_pick	7.71	7.61	17.00	0	True
ORD04 to_drop	5.24	4.71	10.30	0	-
ORD05 to_pick	13.62	14.36	25.40	2	True
ORD05 to_drop	17.33	17.00	32.05	1	-
ORD06 to_pick	12.12	11.55	24.95	0	True
ORD06 to_drop	5.41	5.05	11.60	0	-
HOME to_home	25.54	25.07	38.55	0	-

5b. Dynamic-obstacle timeline (sim/events.log)

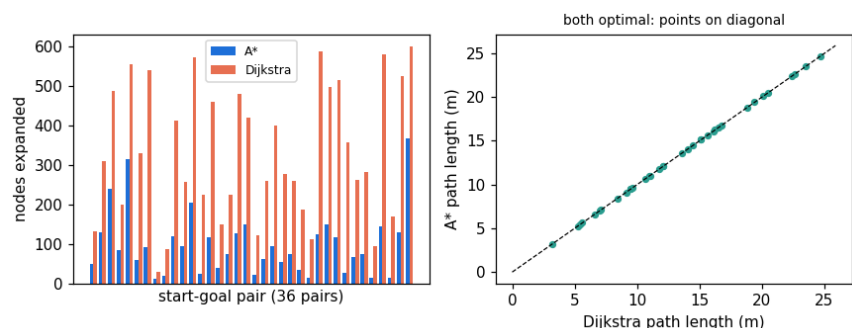
Each box triggers an immediate re-plan when its first cell is confirmed and, typically, a second one a few hundred milliseconds later when the beams grazing its edge commit a neighbouring cell (LiDAR fattening). The robot never had to fall back to clearing stale cells in this run.

t (s)	Event	Cell / detail
77.45	box_spawn	(33, 13)
77.45	replan	(33, 13)
79.65	replan	(33, 12)
140.15	box_spawn	(27, 19)
140.25	replan	(27, 19)
140.80	replan	(27, 18)
167.05	box_spawn	(20, 17)
167.05	replan	(20, 17)

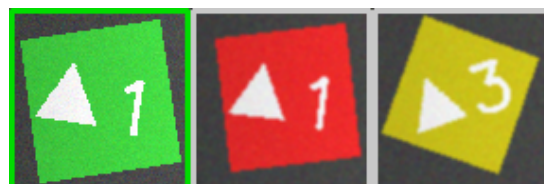
6. A* versus Dijkstra (36 start-goal pairs, static map)

Over all 36 pairs the two planners returned identical path lengths (verified to 1e-9 in the tests), while A* expanded 97.8 nodes on average against 331.4 for Dijkstra (0.45 ms vs 1.28 ms in CPython). The table lists the six pairs with the largest gain; on short aisle-to-aisle hops the gain is small because the octile heuristic is nearly exact there.

From	To	Length A* (m)	Length Dijkstra (m)	Nodes A*	Nodes Dijkstra	ms A* / Dij
P5	D2	13.621	13.621	27	357	0.117 / 1.324
D1	D2	7.0	7.0	15	168	0.091 / 0.618
P2	D1	11.0	11.0	23	223	0.13 / 0.79
P4	D2	5.243	5.243	15	112	0.11 / 0.448
P6	D2	5.414	5.414	13	93	0.061 / 0.354
P1	D2	22.621	22.621	92	540	0.352 / 2.035



7. Figures from the run



Eswar Mahalingam

Robotics & Automation Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

01

Robotics & Automation
Codec Technologies India
Internship 2026

Autonomous Warehouse Robot

A*/Dijkstra planning, LiDAR
re-planning, pure pursuit,
OpenCV item picking

AT A GLANCE

6/6

Orders completed

0

Collisions (whole run)

147 m / 150 m

Driven / planned

5

Re-plans (3 boxes)

98 %

Recogniser (50 items)

98 vs 331

A* vs Dijkstra nodes

270 s

Mission time (sim)

17/17

Unit tests

VERDICT

SIM PASS

TESTS 17/17

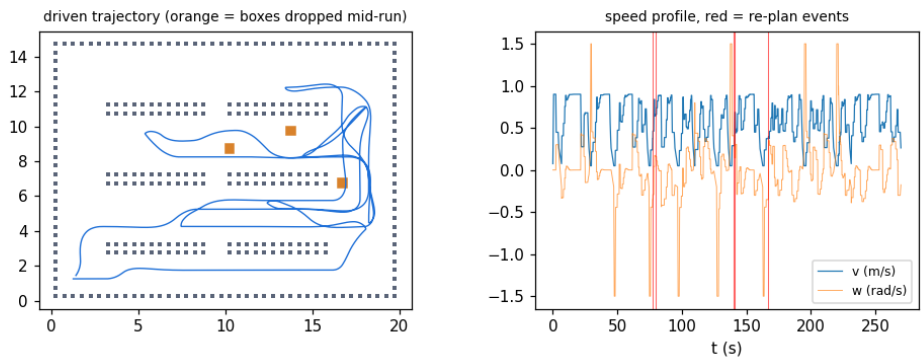
0 COLLISIONS

DECLARED SUBSTITUTION

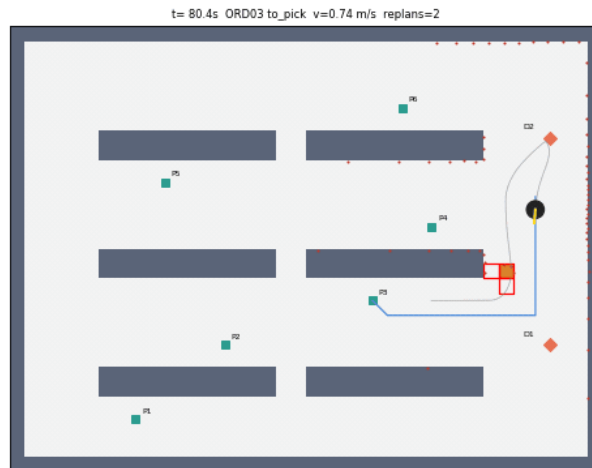
Eswar Mahalingam

Robotics & Automation Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

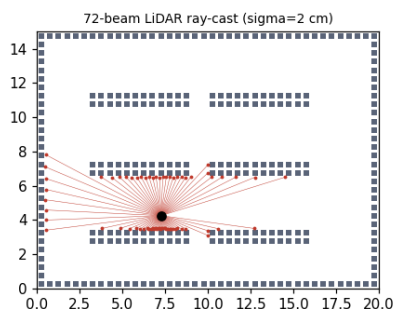
Shelf face presented at P3 for order ORD03 (wanted: green triangle 1). The recogniser read red-triangle-7 | green-triangle-1 | green-square-0 and selected the middle slot (green frame).



Full driven trajectory of the 6-order mission (orange squares = boxes dropped mid-run) and the v/w profiles with re-plan events in red.



GIF frame just after a box (orange) appeared on the route: the LiDAR returns (red dots) are shorter than the static-map prediction, the hit cells are committed to the known map (red outlines) and the blue path has been re-planned around them.



72-beam ray-cast scan from the south-west aisle; every beam is an exact DDA traversal of the occupancy grid plus $N(0, 2 \text{ cm})$ noise on hits.

8. Declared substitutions / not run here

■ **ROS 2 Humble** is not installed in the build container. `ros2_ws/src/warehouse_robot/` contains `package.xml`, `setup.py`, `setup.cfg`, the resource marker, `launch/warehouse_robot.launch.py` and `node.py` (rclpy node, topics listed in its docstring). It was syntax-checked with `py_compile` only. To run: `cd ros2_ws && colcon build && source install/setup.bash && ros2 launch warehouse_robot warehouse_robot.launch.py`.

■ No Gazebo world is needed for this brief; the LiDAR, robot and warehouse are simulated in Python. On a real robot the node consumes `/scan` and `/odom` directly.

9. Learnings / next steps

Two bugs cost me most of the debugging time and both were geometric: pick cells adjacent to a rack face left only 25 cm for a 30 cm robot, and pure pursuit on sparse simplified waypoints with a 1 m look-ahead clipped a rack corner on the return leg. Dense 0.5 m waypoints plus a 0.7 m look-ahead fixed it and the run has been collision-free on five seeds since. Next: replace grid inflation by a proper cost map, add a Kalman-filtered odometry model, and train the digit reader on more fonts so the recogniser holds up beyond the Hershey glyphs it was tested on.